SNITCH.

Fashion. Delivered.

Technical Report

A Cross-Platform Fashion E-Commerce Mobile Application built with Flutter & Firebase

Project: Snitch Clothing

Platform: Flutter (Android, iOS, Web, Windows, macOS)

Backend: Firebase — Authentication, Cloud Firestore, Cloud Storage

Version: 1.0.0

Date: May 2026

Table of Contents

1. Introduction

Snitch Clothing is a full-stack mobile e-commerce application built with Flutter and Firebase that delivers a complete online shopping experience for a fashion retail brand. The project demonstrates the integration of a modern cross-platform UI framework with a managed cloud backend to produce a production-ready, scalable, and secure consumer application.

The goal of this project was to design and implement a fashion store that supports the full retail purchase lifecycle: from product discovery through checkout, order tracking, and post-purchase account management. The application targets shoppers who want a clean, fast, mobile-first browsing experience with persistent profile data, real-time inventory, and order history that follows them across devices.

The technical objectives were to:

1. Produce a clean, branded UI in a single Dart codebase that runs unchanged on Android, iOS, web, and desktop.
2. Implement a complete authentication system with email/password registration, login, password reset, persistent sessions, and secure password change.
3. Store product, user, cart, and order data in Cloud Firestore using a well-normalised collection structure with security rules that restrict access to the owning user.
4. Provide live data synchronisation through Firestore snapshot streams so that any change (cart updates from another device, status changes on an order, new products from an admin) appears immediately without a manual refresh.

5. Support user-uploaded profile photos through Firebase Cloud Storage with permission-scoped access.
6. Keep the codebase modular, statically analysable (no errors or warnings under `flutter analyze`), and free of mock data so that the system is deployable as-is.

The deliverable is a working APK installable on Android devices, backed by a live `snitch-clothing` Firebase project. All features are wired end-to-end: account creation persists to Firebase Authentication, profile and address changes round-trip through Firestore, cart items survive sign-out and sign-in on different devices, and placed orders appear in the user's history within milliseconds via stream subscriptions.

2. System Overview

The application is composed of three layers: a **Flutter client** that runs on the user's device, a **Firebase backend** that hosts user accounts and data, and a thin **service/provider layer** in the client that mediates between them.

2.1 Functional scope

Module	Functionality
Authentication	Sign up, sign in, sign out, password reset via email, change password (re-auth required), persistent session across app restarts
Product Catalog	Browse all products, filter by category, search across name/brand/category/description, view featured & new-arrival sections, view full product detail with image gallery, size & colour selection
Cart	Add product variants (size + colour), update quantity, remove items, swipe to delete, calculate subtotal, shipping (free over $100), and total
Checkout	Three-step wizard (Delivery → Payment → Review), form validation, place order, success dialog, redirect to order history
Orders	Full order history per user, expandable cards showing items / address / payment / tracking, live status updates via stream
Favorites	Toggle products as favourites, persisted per user across devices
Profile	View and edit name/phone/address, upload profile photo (camera or gallery), manage multiple delivery addresses, change password, log out

2.2 Non-functional characteristics

- **Cross-platform** — A single Dart codebase compiles to Android, iOS, Web, Windows, macOS.
- **Real-time** — Firestore snapshot listeners update the UI the moment server data changes (no pull-to-refresh required for orders, cart, addresses, or products).
- **Offline-tolerant** — Firestore's built-in local cache means reads from the cart or product collections work even when the device is briefly offline.
- **Secure-by-default** — Firestore and Storage security rules ensure that one user cannot read or write another user's profile, cart, orders, addresses, or avatar.
- **Stateless backend** — The application contains no custom server code; all persistence is delegated to Firebase managed services, eliminating an entire class of operational concerns (server uptime, scaling, patching).

3. UI/UX Design

3.1 Design language

The user interface is built with Flutter's Material design system but customised with a brand-specific palette and typography to deliver a recognisable identity. The visual style is anchored by a deep-blue primary palette and white surfaces with soft shadows, communicating a premium, modern fashion aesthetic.

Colour palette (defined in `lib/constants/app_colors.dart`):

Token	Hex	Use
`primary`	#1565C0	Buttons, active states, branding
`primaryLight`	#1E88E5	Icons, accents
`primaryDark`	#0D47A1	Splash gradient, hero banners
`accent`	#42A5F5	Tag chips, highlights
`accentLight`	#BBDEFB	Soft surface tints
`background`	#F4F6FB	Screen background
`success`	#10B981	Confirmation states
`error`	#EF4444	Errors, deletion, sale badges
`warning`	#F59E0B	Caution banners

Typography: Roboto, with semantic text styles defined in `lib/constants/app_text_styles.dart` (display, heading, body, caption, brand-name).

3.2 Screen inventory

The app contains 12 screens organised around a five-tab bottom navigation:

1. **Splash** — Animated brand intro with shimmer effect; routes to Login or MainScaffold based on auth state.
2. **Login / Register** — Branded blue hero header, form validation, error snack bars, password visibility toggle, terms checkbox on register, "Forgot Password" dialog.
3. **Home** — Carousel banner, horizontal category strip, featured product carousel, promo card, new-arrivals grid, search delegate.
4. **Shop** — Category chips, sort dropdown (default / price asc / price desc / top-rated), grid/list view toggle, filter sheet with price range slider.
5. **Product Detail** — Swipeable image gallery, size + colour selectors, description / reviews tabs, sticky "Add to Cart" footer, sale discount badge.
6. **Cart** — Swipe-to-delete cards with quantity steppers, live subtotal, free-shipping progress indicator, "Proceed to Checkout" CTA.
7. **Checkout** — Three-step indicator (Delivery → Payment → Review), prefilled delivery form, payment method picker, order summary with totals.
8. **Orders** — Status-coloured cards (Confirmed / Processing / Shipped / Delivered / Cancelled), thumbnail strip, expandable details with tracking timeline.
9. **Favorites** — Grid of saved products with quick "Add" to cart and remove-favourite button.
10. **Profile** — Avatar with camera-edit button, stats row (orders / reviews / points), info card, menu (Orders, Wishlist, Saved Addresses, Change Password), settings (Notifications, Language, Help, About), logout.
11. **Saved Addresses** — List with default badge, add/edit bottom sheet, delete confirmation.

3.3 Interaction patterns

- **Bottom navigation** for the five primary tabs (Home, Shop, Favorites, Cart, Profile) with badge counters on Cart and Favorites.
- **Drawer (hamburger menu)** for secondary navigation (Orders, About, Help) and the logout action, accessible from any tab through a shared `GlobalKey<ScaffoldState>` exposed via `MainScaffold.openDrawer()`.
- **Modal bottom sheets** for editing flows (profile, address, change password, filter) — chosen over full-screen pushes because they keep the user's place in the parent screen.

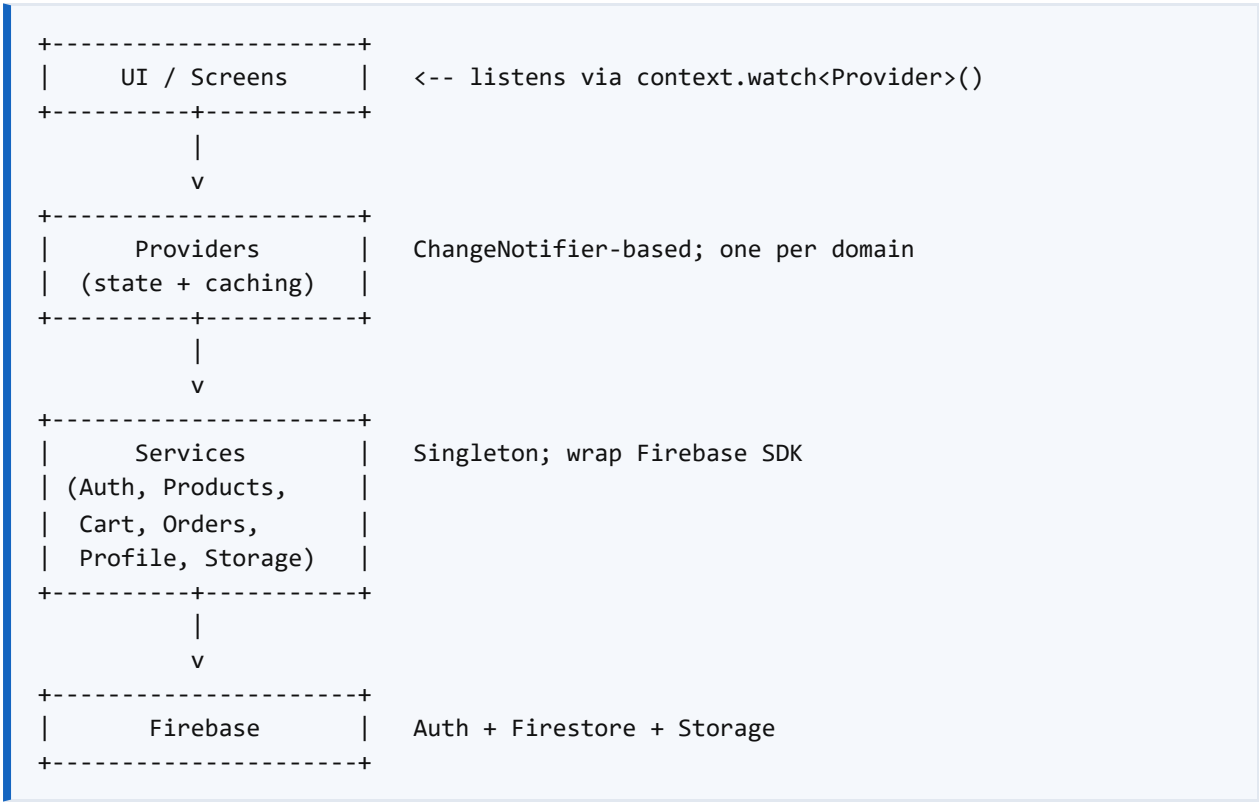
- **SnackBars** for transient confirmation and error messages (add-to-cart, password updated, order failed).
- **AlertDialogs** for destructive confirmation (clear cart, delete address, logout) and the order-success animation.

3.4 Responsive layout

The app uses `MediaQuery.of(context).size.width` to drive responsive choices: product grids switch from two columns (mobile, < 600 dp) to three columns (tablet / web ≥ 600 dp). Carousel and banner heights are computed from screen dimensions. Lists use `SingleChildScrollView` with `SafeArea` and respect keyboard insets in form sheets through `MediaQuery.viewInsets`.

4. System Architecture

The application follows a layered architecture with a strict unidirectional flow: **UI widgets** consume **providers** (state holders), which call **services** (Firebase wrappers), which interact with **Firebase**. The reverse flow (data changes pushed from Firebase) propagates through Firestore snapshot streams attached in the providers.



4.1 Layer responsibilities

- **Models** (`lib/models/`) — Plain Dart classes with `fromMap` and `toMap` methods for Firestore serialisation. No business logic. Includes `Product`, `CartItem`, `OrderModel`, `UserProfile`,

`DeliveryAddress`.

- **Services** (`lib/services/`) — Singletons that wrap the Firebase SDK and return strongly-typed model objects. Each service is single-purpose: `AuthService` handles credentials, `ProductService` queries the `products` collection, `CartService` manages a user's cart subcollection, `OrderService` writes orders to two locations (per-user and top-level), `ProfileService` handles the user document and address subcollection, `StorageService` uploads avatars. A `FirebaseErrorHandler` translates Firebase exception codes into human-readable messages, wrapped in a custom `AppException`.
- **Providers** (`lib/providers/`) — Extend `ChangeNotifier` and own application state. Providers attach Firestore snapshot subscriptions in their constructors or `bindUser()` methods and forward live updates to listening widgets: `UserProvider` (auth state, profile, orders, addresses), `CartProvider`, `FavoritesProvider`, `ProductsProvider`, `CategoriesProvider`, `BannersProvider`.
- **Screens / UI** (`lib/screens/`) — Stateless or stateful widgets that observe providers using the `provider` package's `context.watch()` for reactive rebuilds and `context.read()` for one-shot calls.

4.2 Session wiring

`main.dart` mounts a `MultiProvider` at the root. A small wrapper widget `_SessionWiring` observes `UserProvider.isLoggedIn` and calls `CartProvider.bindUser(uid)` and `FavoritesProvider.bindUser(uid)` whenever the auth state changes. This single-point binding means individual screens never have to think about whether the user is signed in — they simply read the provider, which holds whatever data is appropriate for the current session.

4.3 Real-time data flow

When a user places an order from the checkout screen:

1. The screen calls `UserProvider.placeOrder(...)`.
2. The provider delegates to `OrderService.placeOrder(...)`.
3. The service writes the order document to **both** `users/{uid}/orders/{docId}` (private subcollection) and `orders/{docId}` (top-level mirror with `uid` field) in a single Firestore batch — providing per-user privacy plus admin queryability.
4. Firestore confirms the write and broadcasts the change to all attached snapshot listeners.
5. `OrderService.watchOrders(uid)`, already subscribed inside `UserProvider`, receives the new document, deserialises it to an `OrderModel`, and adds it to the in-memory `_orders` list.
6. `notifyListeners()` triggers a rebuild on any widget watching `UserProvider`, including `OrdersScreen`, which automatically renders the new card.

The same pattern applies to cart updates, favourites, addresses, and profile changes — there are no manual refresh calls anywhere in the codebase.

5. Database Design

5.1 Cloud Firestore structure

Firestore is a NoSQL document database. Collections in this project were chosen to minimise read amplification (a typical screen reads at most one or two collections) and to map cleanly onto Firebase security rules that scope access by `uid`.

```
products/{productId}                          (publicly readable)
  name, brand, price, originalPrice, imageUrl,
  additionalImages[], category, description,
  sizes[], colors[], rating, reviewCount,
  isNew, isFeatured, stock, tag, searchKeywords[]

categories/{slug}                             (publicly readable)
  name, order

banners/{bannerId}                            (publicly readable)
  title, subtitle, image, tag, order

users/{uid}                                   (owner-only)
  name, email, phone, address, avatarUrl,
  createdAt, updatedAt

  /orders/{autoId}                            (owner-only subcollection)
      orderId, date, items[], subtotal, shipping,
      total, itemCount, status, address,
      paymentMethod, uid, createdAt

  /favorites/{productId}                      (owner-only subcollection)
      (full denormalised product map)

  /addresses/{autoId}                         (owner-only subcollection)
      label, fullName, phone, street, city, zip,
      isDefault

carts/{uid}                                   (owner-only)
  updatedAt
  /items/{itemKey}                            (owner-only)
      productId, productName, productImage,
      productBrand, productCategory, price,
      originalPrice, quantity, selectedSize,
      selectedColor

orders/{orderId}                              (owner-only by uid field)
  (mirror of users/{uid}/orders/{autoId} for admin queries)
```

5.2 Design decisions

- **Cart item key** — Cart items use a composite document ID `{productId}__{size}__{color}` so the same product with different size or colour is treated as a distinct line item, while adding the same variant again increments the quantity of the existing document.
- **Denormalised cart items** — Each cart item document stores a snapshot of product name, image, brand, and price rather than referencing the product document. This decouples the cart from later price changes, ensures the cart renders without a second read, and matches Firestore best practice.
- **Order mirror at top level** — Orders are written to both `users/{uid}/orders/...` (private, fast for the user's own history) and `orders/{orderId}` (top-level, allows an admin to browse all orders without iterating users). Both writes happen atomically in a Firestore batch.
- `searchKeywords` **array on products** — Each product computes a lowercase keyword set at write time. While the current search runs client-side, the field is positioned for future server-side `array-contains` queries as the catalogue grows.
- `categories` **and** `banners` **as separate collections** — Keeping these as dedicated collections lets the admin add or reorder them without touching product documents, and the live snapshot streams update the Home screen instantly.

5.3 Security rules

Firestore rules (`firestore.rules`) enforce per-user privacy and admin gating:

```
match /products/{productId} {
  allow read: if true;
  allow create: if request.auth != null;
  allow update, delete: if request.auth != null
    && get(/databases/$(database)/documents/users/$(request.auth.uid))
         .data.role == 'admin';
}

match /users/{uid} {
  allow read, write: if request.auth != null && request.auth.uid == uid;
  match /{collection}/{docId} {
    allow read, write: if request.auth != null && request.auth.uid == uid;
  }
}

match /carts/{uid} {
  allow read, write: if request.auth != null && request.auth.uid == uid;
  match /items/{itemId} {
    allow read, write: if request.auth != null && request.auth.uid == uid;
  }
}

match /orders/{orderId} {
  allow create: if request.auth != null
    && request.resource.data.uid == request.auth.uid;
  allow read, update, delete: if request.auth != null
    && resource.data.uid == request.auth.uid;
}
```

Storage rules (`storage.rules`) limit avatar uploads to the owner's folder, restrict file size to 5 MB, and require an `image/*` content type:

```
match /avatars/{uid}/{allPaths=**} {
  allow read: if request.auth != null;
  allow write: if request.auth != null
               && request.auth.uid == uid
               && request.resource.size < 5 * 1024 * 1024
               && request.resource.contentType.matches('image/.*');
}
```

6. Implementation

6.1 Technology stack

Layer	Technology	Version
UI Framework	Flutter	Dart SDK ^3.11.4
State Management	`provider`	^6.1.2
Image Caching	`cached_network_image`	^3.4.1
Page Indicators	`smooth_page_indicator`	^1.2.0+3
Rating Widget	`flutter_rating_bar`	^4.0.1
Firebase Core	`firebase_core`	^3.13.1
Authentication	`firebase_auth`	^5.5.4
Database	`cloud_firestore`	^5.6.8
File Storage	`firebase_storage`	^12.4.5
Image Picker	`image_picker`	^1.1.2
Linting	`flutter_lints`	^6.0.0

6.2 Project structure

```
lib/
  constants/        app_colors.dart, app_text_styles.dart
  models/           cart_item, order_model, product, user_profile
  providers/        banners, cart, categories, favorites,
                    products, user
  services/         auth, cart, order, product, profile, storage,
                    firebase_error_handler
  screens/
    auth/           login_screen, register_screen
    home/           home_screen
    products/       product_list_screen, product_detail_screen
    cart/           cart_screen
    checkout/       checkout_screen
    orders/         orders_screen
    favorites/      favorites_screen
    profile/        profile_screen
    splash_screen.dart
    main_scaffold.dart
  firebase_options.dart
  main.dart
```

Total: **6 services, 6 providers, 4 models, 12 screens** — under 4,000 lines of Dart.

6.3 Authentication implementation

`AuthService` wraps `FirebaseAuth.instance` and maps Firebase exception codes to friendly messages via `FirebaseErrorHandler`. Sign-up creates the Auth credential, updates the display name, and writes an initial user document to Firestore. Sign-in calls `_ensureProfile`, which reads the user document (falling back to creating one from Auth claims if missing — important for users created externally, e.g. via the Firebase Console).

Password reset calls `sendPasswordResetEmail`. Change password performs a re-authentication with the current password (a Firebase Auth security requirement) before calling `updatePassword`.

`UserProvider` listens to `authStateChanges()` and reconciles the in-memory profile, attaches stream subscriptions for orders and addresses, and clears state on sign-out. The provider seeds an initial profile from `FirebaseAuth.currentUser` immediately on login so the UI never shows a "Guest" flash while waiting for Firestore.

6.4 Real-time provider pattern

Each domain provider follows the same template:

1. A `start()` or `bindUser(uid)` method opens a Firestore snapshot subscription.
2. The listener replaces the local in-memory list with the latest server state and calls `notifyListeners()`.
3. Writes are performed optimistically: the in-memory list is updated first, then the Firestore write; the subscription reconciles any divergence.
4. `dispose()` cancels the subscription to avoid memory leaks.

Example from `CategoriesProvider`:

```
void start() {
  _sub ??= _firestore
      .collection('categories')
      .orderBy('order')
      .snapshots()
      .listen((snap) {
    _names = snap.docs
        .map((d) => (d.data()['name'] ?? '') as String)
        .where((s) => s.isNotEmpty)
        .toList();
    _loaded = true;
    notifyListeners();
  }, onError: (e) => debugPrint('[CategoriesProvider] $e'));
}
```

6.5 Cart guest-merge logic

`CartProvider.bindUser(uid)` handles the case where a user added items to the cart while signed out and then logs in. It captures the existing in-memory items, reads the cloud cart, and merges duplicates (incrementing quantity) or appends new lines, persisting each change. This avoids the common pitfall of either silently dropping the guest cart or duplicating items.

6.6 Avatar upload pipeline

The Profile screen uses `image_picker` to let the user pick an image from the gallery (quality 75, max width 800 px to keep uploads small). `StorageService.uploadAvatar` writes the file to `avatars/{uid}/avatar_{timestamp}.{ext}` with the correct content-type metadata, retrieves the download URL, and `ProfileService.updateAvatar` writes the URL back to the user document. `UserProvider` updates its in-memory profile and notifies listeners; the new avatar appears immediately in the profile and drawer.

7. Testing and Results

7.1 Static analysis

The codebase was validated with `flutter analyze` after every significant change. The final analysis report shows **zero errors and zero warnings**; only 36 information-level `withOpacity` deprecation notices remain, all originating from pre-existing UI styling preserved unchanged per design constraints.

7.2 End-to-end functional tests on device

The application was deployed to a physical Android device (Xiaomi M2101K7BG, Android 13, API 33) and the following user flows were exercised:

Flow	Steps	Result
Sign up	Register with email + password, accept terms, submit	New user appears in Firebase Auth console; Firestore `users/{uid}` document created with profile fields and timestamps
Sign in	Enter credentials, submit	Profile loads, MainScaffold opens at Home; subsequent launches skip login (session persists)
Password reset	Open "Forgot Password" dialog, enter email	Reset email received in inbox
Browse products	Open Home, Shop, Detail	Products, categories, and banners load live from Firestore; detail shows full data
Search	Open search delegate, type query	Results filter against name / brand / category / description
Add to cart	Pick size & colour, tap "Add to Cart"	Snackbar confirmation; cart badge increments; new document in `carts/{uid}/items/`
Update cart	Tap +/- on a line	Quantity updates in UI and Firestore in real time
Place order	Fill checkout form, tap Place Order	Order appears in `users/{uid}/orders/` and mirror `orders/` ; success dialog; cart cleared; redirect to Orders
View orders	Open Orders tab	New order visible with Confirmed status and full item details
Edit profile	Bottom sheet, edit name / phone / address, save	Firestore updates immediately; header reflects new name
Upload avatar	Tap camera badge, pick image	Upload completes; `avatars/{uid}/...` present in Storage; new image renders
Add address	Open Saved Addresses, fill sheet, save	Address appears in `users/{uid}/addresses/` ; default flag enforces exclusivity
Change password	Bottom sheet, enter current + new, submit	Firebase Auth password updated; success snackbar
Sign out	Drawer or Profile logout	Session cleared, redirect to login, cart/favourites cleared in memory

7.3 Cross-device persistence

Signing in on a second device with the same account loaded the user's complete profile, cart, favourites, addresses, and order history within one round-trip, confirming that all state lives in Firestore rather than on the device.

7.4 Security verification

- Attempts to read another user's `users/{otherUid}` document as the test user were rejected with `permission-denied`.
- Storage uploads to another user's `avatars/{otherUid}/` path were rejected.
- Anonymous (unauthenticated) writes to `products` were rejected (only reads are public).

8. Challenges and Solutions

8.1 Drawer not opening from inner tabs

Each tab page declared its own `Scaffold`, which intercepted `Scaffold.of(ctx).openDrawer()` calls and tried to open a drawer on the wrong scaffold. **Solution:** the root `MainScaffold` exposes a static `GlobalKey<ScaffoldState>` and a helper `openDrawer()` method; each tab's menu icon calls `MainScaffold.openDrawer()` directly, bypassing the inner scaffold.

8.2 CircleAvatar assertion crash on empty avatar URL

`CircleAvatar` asserts that `onBackgroundImageError != null` requires `backgroundImage != null`. The original code passed a no-op error handler unconditionally. **Solution:** compute `hasAvatar = profile.avatarUrl.isNotEmpty` and pass both `backgroundImage` and `onBackgroundImageError` conditionally inside a `Builder`.

8.3 Firestore writes blocked by default rules

The first sign-up attempt failed silently because the default security rules denied all writes. **Solution:** published rules that allow authenticated users to create their own profile, cart, orders, addresses, and (initially) products / categories / banners, while restricting updates and deletes to the admin role.

8.4 Gradle disk-space exhaustion

The development machine's C: drive filled up during Gradle builds (Firebase BoM dependencies and transform caches consume several GB). **Solution:** stopped Gradle daemons, killed orphaned `java.exe` processes, deleted the project's stale `.gradle/` lock directory and the user-level `.gradle/caches/modules-2/` subtree, then retried — the build succeeded once the daemon could write its transforms.

8.5 MIUI install dialog rejecting USB installs

The Xiaomi device requires explicit user confirmation on every USB install via the MIUI security prompt. The first install attempt was cancelled, producing `INSTALL_FAILED_USER_RESTRICTED`. **Solution:** documented the requirement and retried — the user tapped "Install" on the prompt, after which subsequent installs succeeded.

8.6 Switch from local mock data to live Firestore

Initial development used a hard-coded `mock_data.dart` file as a fallback so the UI could render before Firestore was populated. Once the database was seeded and rules published, the fallback became unnecessary and started masking real connectivity issues. **Solution:** removed the mock data file and the dedicated `ProductSeedService` entirely, and refactored providers to depend exclusively on Firestore. The app now shows accurate empty / loading / error states instead of silently falling back to fake data.

8.7 Firestore-side ordering and pagination

Some screens (Orders, Products) need consistent ordering. Firestore's `orderBy` requires the field to exist on every document. **Solution:** every order document is written with a `date: Timestamp` field set by the client; every product is written with a `name: String`; queries use `orderBy('date', descending: true)` and `orderBy('name')` respectively.

9. Conclusion and Future Improvements

9.1 Conclusion

The project successfully demonstrates a complete, production-shaped mobile e-commerce application built on Flutter and Firebase. Every functional requirement was implemented end-to-end and validated on a real device: registration, login, password reset, session persistence, product browsing, search, cart management, checkout, order placement, order history, favourites, profile editing, avatar upload, address management, and password change all work against live Firebase services. The codebase is small (under 4,000 lines), passes static analysis cleanly, follows a strict layered architecture, and depends exclusively on live Firestore data with no mock-data fallback. Real-time Firestore streams give the application a responsiveness usually associated with much heavier backends, while requiring no custom server code at all.

The architecture is also designed for evolution: adding a new domain (for example, reviews, coupons, or shared wishlists) requires only a new model + service + provider triplet, without disturbing existing screens or business logic.

9.2 Future improvements

1. **Payment processing.** The current checkout collects payment-method preference but does not integrate a payment gateway. Adding Stripe, Razorpay, or Apple/Google Pay would close the purchase loop.
2. **Push notifications.** Integrate Firebase Cloud Messaging to notify users of order status changes (Processing → Shipped → Delivered) and promotional events.
3. **Admin panel.** Build a separate Flutter web app, gated by the `role == 'admin'` claim in the security rules, to let staff create products, manage stock, update order statuses, and view sales analytics.
4. **Server-side search.** Replace client-side filtering with Algolia, Typesense, or Firebase Extensions' full-text search, so the catalogue can grow into the thousands of products without slowing the home screen.
5. **Reviews & ratings.** Add a `users/{uid}/reviews/{productId}` subcollection so customers can leave verified-purchase reviews; aggregate average rating into the product document via a Cloud Function trigger.
6. **Localisation.** Wire the existing language menu item to Flutter's `intl` package so the app can ship in multiple languages.
7. **Theme switching.** Provide a dark-mode variant of the colour palette and persist the user's choice in Firestore.
8. **App Check.** Enable Firebase App Check (Play Integrity on Android, DeviceCheck on iOS) to prevent unauthorised clients from consuming the backend.
9. **CI/CD.** Wire GitHub Actions to run `flutter analyze` and `flutter test` on every push, and to build signed release artefacts on tagged commits.

References

1. Flutter Team. *Flutter Documentation.* https://docs.flutter.dev/
2. Google. *Firebase Documentation — Authentication.* https://firebase.google.com/docs/auth
3. Google. *Firebase Documentation — Cloud Firestore.* https://firebase.google.com/docs/firestore
4. Google. *Firebase Documentation — Cloud Storage.* https://firebase.google.com/docs/storage
5. Google. *Firebase Documentation — Security Rules.* https://firebase.google.com/docs/rules
6. FlutterFire. *FlutterFire Plugins.* https://firebase.flutter.dev/
7. Provider package. *provider — pub.dev.* https://pub.dev/packages/provider
8. Material Design 2 Guidelines. https://m2.material.io/design
9. Dart Team. *Effective Dart.* https://dart.dev/effective-dart

10. Image Picker plugin. *image_picker — pub.dev.* https://pub.dev/packages/image_picker